Gummy UI (`@gummy-ui/ui`)

A **config-driven** React component library. Instead of hand-writing JSX for a CRUD screen, you describe it with three plain-object configs — **model**, **api**, and **container** — and hand them to a `Core` instance. `Core.run()` returns a fully wired data-table + modal-form UI: typed API calls, runtime validation, responsive grid layout, and theming, with no JSX.

The standalone components (Modal, DataTable2, Paper, Button, …) are also exported for direct use, but the declarative engine is the headline feature.

- **Monorepo** managed with Bun workspaces.
- **TypeScript strict, ESM-only** (no CommonJS).
- **Tailwind CSS + Radix UI** for styling and accessible primitives.

Table of contents

Quick start

Prerequisites

- Bun runtime
- Node.js 18+ (peer tooling)

Install & run

```
bun install        # install + dedupe esbuild (postinstall)
bun run dev        # build ui, then run ui (watch) + api (:9000) + example (:3200)
```

Open http://localhost:3200. The example app proxies `/collection` , `/upload` , `/uploads` , `/health` to the API on `:9000` , so leave `VITE_API_URL` empty in dev.

> **Build order matters.** `apps/example` consumes `@gummy-ui/ui` 's built `dist/` . After changing library source you must rebuild the library (or keep it in `--watch`). `bun run dev` handles this for you — prefer it over starting apps individually.

Use in your own app

```
import { Core, DataProvider, ThemeProvider, HttpClientFactory } from "@gummy-ui/ui";
import "@gummy-ui/ui/styles.css"; // REQUIRED — styles ship separately
```

The declarative engine

A screen is assembled from three configs and rendered by `Core`. The configs are linked by name: `api` entries reference `model` names, giving end-to-end typing from endpoint to payload.

```
model  ──►  describes data shapes (request/response/params/query)
api    ──►  describes endpoints, referencing model names by string
container ─► describes responsive layout (rows of breakpoint-spanned bins)
                          │
                          ▼
        new Core(http, model, api, containers).run()  ──►  React elements
```

1. model — data shapes

Named data shapes built from the primitives `"string" | "number" | "boolean" | "integer" | "any"`, plus nested `{ type: "array" | "object", collection: {...} }`. Converted at runtime to TypeBox schemas for validation.

```ts
import type { TModelMaster } from "@gummy-ui/ui";

export const model: TModelMaster = {
  countryRes: {
    data: {
      type: "array",
      collection: { _id: "string", name: "string", code: "string", avatar: "any" },
    },
    status: "number",
    success: "boolean",
    message: "string",
  },
  countryBody: { name: "string", code: "string", flagImage: "string" },
  countryParam: { id: "string" },
  countrySearchQuery: { search: "string" },
};
```

2. api — endpoints

Named endpoints. Each entry's `response` / `body` / `query` / `parameter` keys reference **model names** (strings). URLs use `:param` placeholders matched against the `parameter` model.

```ts
import type { TApiMaster } from "@gummy-ui/ui";

export const api = {
  countries: {
    url: "/collection/get-all",
    description: "Get all countries",
    methods: "POST" as const,
    response: "countryRes",
    body: "countryBody",
    withOptions: false,
  },
  updateCountry: {
    url: "/collection/update/:id",
    description: "Update country",
    methods: "PATCH" as const,
    response: "countryRes",
    body: "countryBody",
    parameter: "countryParam",
    withOptions: false,
  },
} satisfies TApiMaster<typeof model>;
```

3. container — layout

An array of `Container`. Each container holds `bins`; each `Bin` carries breakpoint spans (`sm` / `md` / `lg` / `xl` as `"1".."12"` of a 12-column grid), a `type`, and an `element` describing the field.

```
import type { Container } from "@gummy-ui/ui";

export const containerCountryList: Container[] = [
  {
    id: "country-list",
    name: "country",
    isArray: false,
    bins: [
      {
        sm: "12", md: "12", lg: "12", xl: "12",
        type: "datatable",
        element: {
          name: "countries",
          title: "Countries",
          columns: [
            { accessor: "name", header: "Name", enableSorting: true, enableColumnFilter: true },
            { accessor: "code", header: "Code", enableSorting: true, enableColumnFilter: false },
          ],
          api: { name: "countries" },
          canEdit: true,
          canDelete: true,
        },
      },
    ],
  },
];
```

4. Container API load

Use `load` on a `Container` to fetch data when the container mounts and store the result in a **global state slice** (one zustand store per `key`). Fields inside the container can then read from that slice via `value: { type: "state", key, path }`.

On mount the loader:

1. Resolves `load.api.params` / `query` / `body` (including `type: "url"` values from react-router).
2. Calls the named endpoint from your `api` config.
3. Drills into the response with `load.api.paths` (e.g. `["data"]` to unwrap `{ data: { … } }`).
4. Writes the result under `load.key`.
5. Refetches when resolved URL params change; clears the slice on unmount.

1. Declare the endpoint in `api.ts` (same as any other endpoint):

```
countryDetail: {
  url: "/collection/detail/:id",
  description: "Get a single country by id",
  methods: "GET" as const,
  response: "countryDetailRes",
  parameter: "countryParam", // { id: "string" }
  withOptions: false,
},
```

2. Add `load` to the container and bind fields to the slice:

```
export const containerCountryStateDetail: Container[] = [
  {
    id: "state-detail-1",
    name: "CountryStateDetail",
    isArray: false,
    load: {
      key: "countryDetail", // global-state slice name
      api: {
        name: "countryDetail",       // must match an entry in api.ts
        paths: ["data"],             // drill response.data before storing
```

```
        params: { id: { type: "url", key: "id" } }, // :id from the route
      },
    },
    bins: [
      {
        sm: "12", md: "6", lg: "6", xl: "6",
        type: "textfield",
        element: {
          name: "name",
          label: "Name",
          dataType: "string",
          isRequired: false,
          errorMessage: "",
          // read initial value from the loaded slice
          value: { type: "state", key: "countryDetail", path: "name" },
        },
      },
      // …more bins bound to countryDetail.* paths
    ],
  },
];
```

3. Route the page so URL params are available. The example app mounts a separate `Core` instance per route and passes `:id` through react-router:

```
<Routes>
  <Route path="/" element={<ListPage />} />
  <Route path="/country/:id" element={<DetailPage />} />
</Routes>
```

Open `/country/1` in the example app to see the state-loader demo (links on the list page).

> **Note:** `load.api` is the runtime **API** config (name + paths + params/query/body), not the `TApiMaster` entry. `params` keys must match the endpoint's `parameter` model field names; `query` / `body` keys match their respective models.

5. Wire it up

`Core` must render inside `<DataProvider>` + `<ThemeProvider>`.

```
import { Core, DataProvider, ThemeProvider, HttpClientFactory, ThemeToggle } from "@gummy-ui/ui";
import "@gummy-ui/ui/styles.css";
import { model } from "./config/country/model";
import { api } from "./config/country/api";
import { containerCountryList } from "./config/country/container";
import { theme, components } from "./config/theme";

const http = new HttpClientFactory(
  import.meta.env.VITE_API_URL ?? "", // base URL
  async () => "",                     // async access-token getter
  "1.0.0",                            // version
  30000                               // timeout (ms)
);

export function App() {
  const ui = new Core(http, model, api, containerCountryList).run();
  return (
    <DataProvider>
      <ThemeProvider theme={theme} components={components}>
        <ThemeToggle />
        {ui}
      </ThemeProvider>
    </DataProvider>
  );
}
```

> To add a new feature screen, create a sibling folder `apps/example/src/config/<feature>/` with `model.ts`, `api.ts`, and `container.ts`, mirroring `config/country/`.

Configuration reference

`HttpClientFactory` constructor

#	Argument	Type	Default	Description
1	`baseUrl`	`string`	—	API base URL. Leave `""` in dev to use the Vite proxy.
2	`getAccessToken`	`() => Promise<string>`	—	Async token getter (e.g. `getAccessToken` from Azure MSAL).
3	`version`	`string`	`"1.0.0"`	App/API version header.
4	`timeout`	`number`	`30000`	Request timeout in ms.
5	`ignoreLoadingRequest`	`IgnoreService[]`	`[]`	Requests excluded from the global loading indicator.
6	`ignoreErrorRequest`	`IgnoreService[]`	`[]`	Requests excluded from global error handling.
7	`unwrap`	`Function`	—	Optional response unwrapper.
8	`onError`	`ErrorFunction`	—	Global error callback.
9	`onLog`	`LogFunction`	—	Global request logger.

`Core` constructor

#	Argument	Type	Description
1	`http`	`HttpClientFactory`	The configured HTTP client.
2	`model`	`TModelMaster`	Named data shapes.
3	`api`	`TApiMaster<typeof model>`	Named endpoints referencing model names.
4	`containers`	`Container[]`	Responsive layout.

`Core.run()` → returns the rendered React element tree.

`model` field values (`TModel`)

Value	Meaning
`"string"`	String field
`"number"`	Number field
`"integer"`	Integer field
`"boolean"`	Boolean field
`"any"`	Untyped / arbitrary value
`{ type: "object", collection: {…} }`	Nested object
`{ type: "array", collection: {…} | "string" }`	Array of objects or primitives

`api` entry (`TApiMaster` value)

Property	Type	Required	Description
`url`	`string`	✔	Endpoint path; supports `:param` placeholders.
`methods`	`"GET" | "POST" | "PUT" | "PATCH" | "DELETE"`	✔	HTTP method.
`description`	`string`	✔	Human-readable description.

Property	Type	Required	Description
`response`	model name	✔	Model describing the response payload.
`body`	model name	—	Model for the request body.
`query`	model name	—	Model for query-string params.
`parameter`	model name	—	Model for `:param` path values.
`withOptions`	`boolean`	✔	Whether the caller passes per-request options.

`Container`

Property	Type	Default	Description
`id`	`string`	—	Unique container id.
`name`	`string`	—	Logical name (binds to data context).
`isArray`	`boolean`	—	Whether the bound data is a collection.
`bins`	`Bin[]`	—	Grid items.
`contextData`	`string`	—	Data-context key to bind.
`gap`	`string | number`	`"2"`	Grid gap — Tailwind scale key or CSS length.
`justifyItems`	`"start" | "end" | "center" | "stretch"`	—	Grid `justify-items`.
`alignItems`	`"start" | "end" | "center" | "stretch"`	—	Grid `align-items`.
`justifyContent` / `alignContent`	`ContainerGridContent`	—	Grid content distribution.
`gridAutoFlow`	`ContainerGridAutoFlow`	—	Grid auto-flow.
`surface`	`boolean | ContainerSurface`	—	Themed panel wrapping the grid. `true` = defaults.
`load`	`ContainerLoad`	—	Fetch on mount; store response in global state under `load.key`.

`ContainerLoad`

Property	Type	Required	Description
`key`	`string`	✔	Global-state slice name. Must be unique per concurrent loader.
`api`	`API`	✔	Runtime API call config (see below).

Runtime `API` (element / loader binding)

Used on `Container.load`, DataTable `api`, button actions, etc. References a named endpoint from `TApiMaster` and supplies runtime args.

Property	Type	Required	Description
`name`	`string`	✔	Endpoint name from `api.ts`.
`paths`	`string[]`	—	Drill into the response before storing or displaying (e.g. `["data"]`, `["data", "items"]`).
`params`	`Record<string, DataValue>`	—	Path-param values (`:id` placeholders). Keys match the endpoint's `parameter` model.
`query`	`Record<string, DataValue>`	—	Query-string values. Keys match the endpoint's `query` model.
`body`	`Record<string, DataValue>`	—	Request-body values. Keys match the endpoint's `body` model.
`pagination`	`DataTablePagination`	—	DataTable only — server-side paging config.

`DataValue`

Resolves a concrete value for API args or field `value` bindings.

Property	Type	Required	Description
`type`	`"value" | "state" | "url" | "variable" | "observe" | "selectedRow"`	✔	Source kind.
`key`	`string | "none"`	✔	Slice name (`state`), URL param/query name (`url`), etc.
`path`	`string`	—	Lodash path into a `state` object (e.g. `"name"`, `"address.city"`). Omit to use the whole slice.
`value`	`any`	—	Literal when `type: "value"` .
`source`	`"param" | "query"`	—	For `type: "url"` : read from route param (default) or query string.

Common patterns:

Pattern	Config	Use case
Route param	`{ type: "url", key: "id" }`	`:id` in the URL → API path param
Query string	`{ type: "url", key: "q", source: "query" }`	`?q=…` → API query param
Literal	`{ type: "value", key: "none", value: 10 }`	Fixed body/query value
Loaded field	`{ type: "state", key: "countryDetail", path: "name" }`	Bind input to data fetched by `load`

`Bin`

Property	Type	Required	Description
`sm` / `md` / `lg` / `xl`	`"1".."12"`	✔	Column span per breakpoint (12-col grid).
`type`	`BinType`	✔	Which element to render (see below).
`element`	`TElement`	—	The field/element config.
`container`	`Container`	—	Nested container (for `type: "container"` / `"tab"` / `"paper"`).
`condition`	`CondExpression`	—	Conditional render/visibility expression.
`align`	`"start" | "center" | "end"`	—	Self alignment shorthand.
`justifySelf` / `alignSelf`	grid self values	—	Per-item grid alignment.

`BinType` values: `hidden` , `textfield` , `textarea` , `select` , `checkbox` , `radio` , `autocomplete` , `multiAutocomplete` , `datepicker` , `daterangepicker` , `datetimepicker` , `datatable` , `datatableeditable` , `text` , `typography` , `avatar` , `uploadimage` , `uploadfile` , `button` , `modal` , `tab` , `paper` , `divider` , `container` , `empty` .

`ContainerSurface`

Property	Type	Default	Description
`background`	`boolean`	`true`	Paint a panel background (white / `gray-900` dark).
`border`	`boolean`	`true`	Draw a border.
`accentBorder`	`boolean`	`false`	Tint border with the theme accent.
`radius`	`"none" | "sm" | "md" | "lg" | "xl"`	`"md"`	Corner radius.
`padding`	`"0" | "2" | "3" | "4" | "5" | "6" | "8"`	`"4"`	Inner padding (Tailwind key).
`shadow`	`"none" | "sm" | "md" | "lg"`	`"sm"`	Drop shadow.
`title`	`string`	—	Heading rendered at the top.
`accentTitle`	`boolean`	`false`	Tint the title with the accent color.

Element examples

`datatable` **element (** `DataTableElement` **)**

Property	Type	Description
`name`	`string`	Element id.
`title`	`string`	Table heading.
`columns`	`ColumnDef[]`	Column defs (`accessor` , `header` , `enableSorting` , `enableColumnFilter` , `isEditable?` , `align?` , `useDateFormat?`).
`api`	`API`	Read endpoint reference (`{ name }`).
`apiDeleteInfo`	`APIDelete`	Delete endpoint reference.
`modalContainer`	`Container`	Layout of the edit/create modal form.
`modalMaxWidth` / `modalMinWidth` / `modalMaxHeight`	`string`	Modal sizing.
`canEdit` / `canDelete`	`boolean`	Toggle row actions.

`textfield` **element (** `TextFieldElement` **)** — `name` , `dataType` , `isRequired` , `errorMessage` , plus all `TextField` props (`label` , `placeholder` , `regex` , `variant` , `size` , `radius` , …).

`DataType` **values** (input semantics): `text` , `number` , `password` , `email` , `tel` , `url` , `search` , `date` , `datetime-local` , `time` , `month` , `week` , `hidden` .

Theming

Theming has two mechanisms, both configured via `<ThemeProvider>` :

1. **Global Radix tokens** — `accentColor` , `appearance` (light/dark), `radius` , `panelBackground` . The accent exposes CSS vars `--accent-1..12` and `--accent-contrast` that re-tint automatically for dark mode.
2. **Per-component color overrides** — the `components` map (`button` , `dataTable`).

```
// apps/example/src/config/theme.ts
import type { ThemeComponents, ThemeProps } from "@gummy-ui/ui";

export const theme: ThemeProps = { accentColor: "teal" };

export const components: ThemeComponents = {
  button: { color: "teal" },
  dataTable: {
    headerColor: "teal",
    rowHoverColor: "teal",
    editButtonColor: "teal",
    deleteButtonColor: "red",
  },
};
```

`ThemeProviderProps`

Property	Type	Description
`theme`	`ThemeProps`	Radix global tokens (`accentColor` , `appearance` , `radius` , `panelBackground`). Omitted keys fall back to defaults (`appearance: "light"` , `accentColor: "blue"` , `radius: "small"`).
`components`	`ThemeComponents`	Per-component color overrides.
`className`	`string`	Wrapper class.
`children`	`ReactNode`	App subtree.

`ThemeComponents`

Key	Fields	Notes
`button`	`color`	Accent color for buttons.
`dataTable`	`headerColor`, `headerHoverColor`, `paginationButtonColor`, `paginationButtonHoverColor`, `rowHoverColor`, `editButtonColor`, `deleteButtonColor`	**All optional.** Unset roles follow the theme accent (`--accent-*`) and flip with dark mode. Setting a **named color** pins a static light tint for that role.

Light / dark mode

`ThemeProvider` toggles a `dark` class on `<html>` and persists the choice to `localStorage` (`gummy-ui-appearance`). Drop in `<ThemeToggle />` anywhere inside the provider to let users flip it.

> **Rule when authoring components:** never hardcode `#fff` /hex. Use `dark:` Tailwind classes (`bg-white dark:bg-gray-900`) for surfaces and the `--accent-*` vars (`bg-[var(--accent-9)]`, `text-[var(--accent-contrast)]`) for primary surfaces. See the "Theming rule" in `CLAUDE.md` .

Standalone components

All of the following are exported from `@gummy-ui/ui` for direct JSX use.

`Paper` props

Prop	Type	Default	Description
`elevation`	`number` (0–24)	`1`	Shadow depth (MUI-like). Ignored when `variant="outlined"` .
`variant`	`"elevation" | "outlined"`	`"elevation"`	Shadow vs 1px border.
`square`	`boolean`	`false`	Disable rounded corners.
`className` / `style`	—	—	Pass-through.

`Divider` props

Prop	Type	Default	Description
`variant`	`"fullWidth" | "inset" | "middle"`	`"fullWidth"`	Edge-to-edge, left-indented, or both-indented.
`spacing`	`number | string`	`"8px"`	Margin above/below. Number → px.
`className` / `style`	—	—	Pass-through.

`Modal` props

Prop	Type	Default	Description
`id`	`string`	—	Required modal id (used by the engine to open it).
`title`	`string`	—	Header title.
`trigger`	`JSX.Element`	—	Element that opens the modal.
`children`	`ReactNode`	—	Modal body.
`open`	`boolean`	—	Controlled open state.
`onOpenChange`	`(open: boolean) => void`	—	Open-state callback.
`maxWidth` / `minWidth` / `maxHeight`	`string`	—	Sizing.

Prop	Type	Default	Description
`hiddenTrigger`	`boolean`	—	Hide the trigger (open imperatively).
`isHideTitleLine`	`boolean`	—	Hide the divider under the title.

`ThemeToggle` props

Prop	Type	Default	Description
`size`	`"1" | "2" | "3" | "4"`	`"2"`	Radix IconButton size.
`variant`	`"classic" | "solid" | "soft" | "surface" | "outline" | "ghost"`	`"soft"`	Button variant.
`className`	`string`	—	Pass-through.

`Button` props (declarative `ButtonProps`)

Prop	Type	Description
`label`	`string`	Button text (required).
`variant`	`"solid" | "outline" | "ghost" | "link"`	Visual style.
`color`	accent color	Override accent.
`icon`	icon name	Leading icon.
`actions`	`ButtonAction[]`	Declarative actions (call API, open modal, reload table…).
`api` / `apiInfo`	—	API binding for action buttons.
`modalId`	`string`	Modal to open.
`reloadDataTable`	`string`	DataTable id to refresh after action.
`snackbarSuccess` / `snackbarError`	—	Toasts on success/error (`"$exception"` shows the thrown error).
`confirmBox`	`ConfirmBoxElement`	Confirmation dialog before action.

`TextField` props

Prop	Type	Default	Description
`dataType`	`DataType`	—	Input semantics (required).
`label`	`string`	—	Field label.
`placeholder`	`string`	—	Placeholder.
`helperText`	`string`	—	Helper text below the field.
`error`	`boolean`	—	Error state.
`errorMessage`	`string`	—	Error message.
`variant`	`"classic" | "surface" | "soft"`	—	Radix variant.
`size`	`"1" | "2" | "3"`	—	Field size.
`radius`	`"none" | "small" | "medium" | "large" | "full"`	—	Corner radius.
`isFullWidth`	`boolean`	—	Stretch to container width.
`width`	`number`	—	Fixed width (px).
`isFixedHeight`	`boolean`	—	Keep height stable when showing errors.
`regex`	`string | RegExp`	—	Reject keystrokes producing a non-matching value.

Prop	Type	Default	Description
`regexErrorMessage`	`string`	`"Invalid character"`	Hint shown when a keystroke is rejected.

> Other exported components include `DataTable2`, `DataTableEditable`, `Autocomplete2`, `MultiAutocomplete`, `Snackbar`, `ConfirmBox`, `Popover`, `Avatar`, `Text`, `Typography`, `Icon`, `Tab`, `DateRangePicker`, `DateTimePicker`, `UploadImage`, `UploadFile`. The full public surface is curated in `packages/ui/src/index.ts`.

Project structure

```
ui-toolkit/
├── packages/
│   └── ui/                      # @gummy-ui/ui — the library (built with tsup)
│       └── src/
│           ├── components/
│           │   ├── core/        # the declarative engine (Core, builders, per-component config)
│           │   ├── form/        # form field components
│           │   ├── context/     # DataProvider, ThemeProvider, LoadingProvider
│           │   └── @types/      # the config contract (Bin, Container, element types)
│           ├── model/           # TModel → TypeBox/Zod converters
│           ├── api/             # HttpClientFactory, ApiFactory, APIMaster
│           ├── auth/azure/      # Azure AD (MSAL) token integration
│           ├── util/            # helpers + constant/colors.ts (theme color maps)
│           └── styles.css       # ships RAW, imported as @gummy-ui/ui/styles.css
├── apps/
│   ├── example/                 # Vite demo (:3200) — config/country/ shows the engine
│   └── api/                     # Hono mock backend (:9000)
└── scripts/dev.mjs              # dev orchestrator
```

Development & build

Command	What it does
`bun install`	Install deps; postinstall dedupes esbuild.
`bun run dev`	Build ui, then run ui (watch) + api (:9000) + example (:3200).
`bun run example`	Example app only (Vite, :3200).
`bun run api`	API server only (Hono, :9000).
`bun run build`	Build all packages.
`bun run --filter @gummy-ui/ui build`	Build the library only (tsup).
`bun run --filter @gummy-ui/ui typecheck`	`tsc --noEmit` — run before committing.
`bun test packages/ui/src/model/master.test.ts`	Run a single test (`bun:test`).

Notes

- **`esbuild` is pinned to `0.27.7`** via a root `overrides`, and `postinstall` deletes nested copies under `vite` / `tsup`. The example's `vite.config.mjs` force-enables `supported.destructuring`. Don't change the esbuild version or remove these workarounds — it breaks the build.
- **Styles ship raw.** `@gummy-ui/ui/styles.css` is processed by the consumer's PostCSS, so `@apply dark:` and `.dark` selectors resolve at the consumer — no library rebuild needed for `styles.css` changes.
- **Tailwind only emits classes that appear as literal strings** in scanned source. Runtime-built class names are dropped — keep accent/dark classes as literal constants or static lookup maps.

License

Private. See `package.json`.